FastAPI Learn Advanced User Guide

Templates

You can use any template engine you want with **FastAPI**.

A common choice is Jinja 2 , the same one used by Flask and other tools.

There are utilities to configure it easily that you can use directly in your **FastAPI** application (provided by Starlette).

Install dependencies

Make sure you create a virtual environment, activate it, and install `jinja2`:

```bash
fast →
$ pip install jinja2

57%
```

Using `Jinja2Templates`

- Import `Jinja2Templates`.
- Create a `templates` object that you can reuse later.
- Declare a `Request` parameter in the *path operation* that will return a template.
- Use the `templates` you created to render and return a `TemplateResponse`, pass the name of the template, the request object, and a "context" dictionary with key-value pairs to be used inside of the Jinja 2 template.

Python 3.10+

```python
from fastapi import FastAPI, Request
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates

app = FastAPI()

app.mount("/static", StaticFiles(directory="static"), name="static")


templates = Jinja2Templates(directory="templates")


@app.get("/items/{id}", response_class=HTMLResponse)
async def read_item(request: Request, id: str):
    return templates.TemplateResponse(
        request=request, name="item.html", context={"id": id}
```

Ask AI

```
    )
```

Note

Before FastAPI `0.108.0`, Starlette `0.29.0`, the `name` was the first parameter.

Also, before that, in previous versions, the `request` object was passed as part of the key-value pairs in the context for Jinja `2`.

Tip

By declaring `response_class=HTMLResponse` the docs UI will be able to know that the response will be HTML.

Technical Details

You could also use `from starlette.templating import Jinja2Templates`.

FastAPI provides the same `starlette.templating` as `fastapi.templating` just as a convenience for you, the developer. But most of the available responses come directly from Starlette. The same with `Request` and `StaticFiles`.

Writing templates

Then you can write a template at `templates/item.html` with, for example:

```
<html>
<head>
    <title>Item Details</title>
    <link href="{{ url_for('static', path='/styles.css') }}" rel="stylesheet">
</head>
<body>
    <h1><a href="{{ url_for('read_item', id=id) }}">Item ID: {{ id }}</a></h1>
</body>
</html>
```

Template Context Values

In the HTML that contains:

```
Item ID: {{ id }}
```

...it will show the `id` taken from the "context" `dict` you passed:

```
{"id": id}
```

For example, with an ID of `42`, this would render:

```
Item ID: 42
```

Template `url_for` Arguments

You can also use `url_for()` inside of the template, it takes as arguments the same arguments that would be used by your *path operation function*.

So, the section with:

```
<a href="{{ url_for('read_item', id=id) }}">
```

...will generate a link to the same URL that would be handled by the *path operation function* `read_item(id=id)`.

For example, with an ID of `42`, this would render:

```
<a href="/items/42">
```

Templates and static files

You can also use `url_for()` inside of the template, and use it, for example, with the `StaticFiles` you mounted with the `name="static"`.

```
<html>
<head>
    <title>Item Details</title>
    <link href="{{ url_for('static', path='/styles.css') }}" rel="stylesheet">
</head>
<body>
    <h1><a href="{{ url_for('read_item', id=id) }}">Item ID: {{ id }}</a></h1>
</body>
</html>
```

In this example, it would link to a CSS file at `static/styles.css` with:

```
h1 {
    color: green;
}
```

And because you are using `StaticFiles`, that CSS file would be served automatically by your **FastAPI** application at the URL `/static/styles.css`.

More details

For more details, including how to test templates, check Starlette's docs on templates.